

Microservices Architecture Blueprint and Roadmap

Overview

This project will be built as a **microservices-based platform** using **Node.js (Express)** for backend services and **Angular 20** (with PrimeNG and Tailwind CSS) for front-end applications. The system will support both **single-tenant and multi-tenant** configurations, and will use **PostgreSQL** as the primary database for each service. Key infrastructure components include **Kafka** for inter-service messaging and **Redis** for caching and other optimizations. The goal is to have a modular, scalable architecture with **Single Sign-On (SSO)** style authentication via **JWT tokens**, enabling users to log in once and access multiple services seamlessly. The **core (home) service** will act as the central hub (and identity provider) for the platform, coordinating authentication and providing a home dashboard, while distinct microservices (e.g., a Form Builder, a URL Shortener, etc.) handle specific functionality. The entire setup will be designed with **containerization** in mind for easy deployment (Docker/Kubernetes), while keeping local development simple (running services without Docker). A high-level summary of the stack and features includes:

- **Backend:** Node.js + Express microservices (independently deployable services) ¹, each with its own database (ensuring loose coupling of data ²) and specific domain logic.
- **Front-end:** Multiple Angular 20 applications (one per service or domain), with a **Shared UI Components Library** (for reuse of common components across apps) to ensure a consistent look and feel.
- **Database:** PostgreSQL for reliable relational storage; structured to support multi-tenancy (either via shared schema with tenant IDs or separate schemas/DBs per tenant as configured ³).
- **Authentication:** Centralized JWT-based SSO. Users authenticate through the core service and receive a JWT token that is trusted by all other services (eliminating the need for each service to have its own login) ⁴. The JWT will carry user identity, tenant, and authorization claims.
- **Inter-service Communication:** Kafka will be used as a **message broker** for publishing and consuming events between services, enabling an event-driven architecture that improves scalability and decoupling ⁵. Redis will be used for caching frequently accessed data and potentially for distributed locks or rate-limiting, improving performance and throughput ⁶.
- **Multi-Tenancy:** The system can be configured for single-tenant mode (one tenant, simplified admin) or multi-tenant mode (multiple customer organizations on one instance). In multi-tenant mode, data is isolated per tenant (e.g., via tenant IDs or schema separation) so that one client's data is not accessible to another ⁷. The approach to data isolation can evolve – starting simple with a tenant identifier in each table, and later expanding to schema-per-tenant or even database-per-tenant for greater isolation ³.
- **DevOps:** CI/CD pipelines will automate testing, building, and deployment of each microservice. In production, each service (and front-end) will be containerized (Docker images) for deployment on a server or Kubernetes cluster. For local development, developers can run services directly (Node and Angular dev servers) without containers for simplicity, while ensuring the system is *docker-ready* when moving to staging/production.

Following is a detailed breakdown of the architecture, components, and a step-by-step **roadmap** for implementation.

System Architecture

Microservices and Their Responsibilities

We will organize the platform into several **independently deployable microservices**, each responsible for a distinct feature set. Initially, we plan the following services:

- **Core Service (Home & Auth)** – *Node.js/Express + Angular*: This is the central service that provides the landing page, authentication (login/logout, user management), and a home dashboard. It can be considered the **API Gateway/BFF** (Backend-for-Frontend) in our architecture, acting as the single entry point for the system ⁸. It authenticates users and issues JWTs, and may provide global features like a navigation portal to other services. In multi-tenant mode, the core service also manages tenant administration (creating new tenant organizations, managing which services a tenant has access to, etc.). The Angular front-end for core will handle the main UI shell (home page, login page, user profile, etc.).
- **Form Builder Service** – *Node.js/Express + Angular*: A separate microservice that provides a form builder tool (for designing and publishing forms). It has its own backend API (for CRUD operations on form templates, questions, responses, etc.) and a dedicated Angular front-end application for the UI (form builder interface). This service stores form data in its own database schema/tables (isolated from other services). It relies on the core service for authentication; users must log in via the core (SSO) to access the form builder. Once authenticated, the form builder front-end uses the JWT to authorize API requests. The service will check the JWT to ensure the user is allowed to use the form builder (based on claims or roles). In a multi-tenant scenario, forms and responses are tagged by tenant so that each tenant's forms are kept separate.
- **Short Link Service** – *Node.js/Express + Angular*: Another microservice providing URL shortening functionality. It has a backend API to create short URLs and redirect to targets, and may have an Angular admin UI for users to view/manage their shortened links. It maintains its own data store for links. Like the form builder, it uses the shared SSO for login (only authenticated users with appropriate access can create/manage short links, though the redirect endpoint for a short URL might be public). The short link service might use **Redis caching** heavily – for example, caching the mapping of short codes to target URLs in Redis to speed up redirects and reduce database hits. It will also validate JWTs on protected endpoints to ensure the user is authorized for this service.
- **(Future) Additional Services** – The architecture is designed to easily accommodate more microservices later (e.g., an **Email Notification Service** to send emails or an **Analytics Service** to gather usage stats). Each new service will follow the same pattern: its own Node.js backend, its own Angular front-end (if interactive UI is needed), its own database tables, and integration into the authentication and messaging ecosystem. We will document later in this blueprint how to add a new service with minimal friction (see the section **Extending the Architecture: Adding New Services**).

Each microservice is self-contained, owning its **own data and logic**, which is a core tenet of microservice design (each service typically has its own separate database to avoid tight coupling ²). Services

communicate with each other **through APIs or asynchronous messages**, never by directly reading each other's databases. This ensures that services remain modular and can be developed and deployed independently.

Communication pattern: For direct client-facing operations, we will use synchronous RESTful APIs. The Angular front-ends will call the respective service's REST endpoints (e.g., the Form Builder Angular app calls the Form Builder service's API). In cases where one service needs data or actions from another, we will favor **event-driven communication** using Kafka (and possibly Redis pub/sub for lighter weight events). For example, when a new user registers via the Core service, the Core could publish a `"user.created"` event to Kafka; other services (if interested) can consume this event – e.g., an Email service could send a welcome email, or the Form Builder service could create some default forms for that user, etc. This decoupling via a message broker ensures that services are not tightly integrated via direct calls, improving scalability and fault tolerance ⁵. Kafka guarantees reliable, at-least-once delivery of events, so if a service is temporarily down, it can catch up on events later without data loss ⁹.

API Gateway consideration: In a robust microservice setup, often an API Gateway is used as a single entry point to route requests to the appropriate service and handle cross-cutting concerns like authentication ¹⁰ ¹¹. In our design, the **Core service** can act as a simple API Gateway/BFF for certain calls – particularly for authentication and perhaps aggregating some data for the home dashboard – but for simplicity, front-end apps might also call other service APIs directly. Initially, we can allow direct API calls from the micro-frontend to its microservice (using CORS with appropriate tokens). As the system evolves, we could introduce an API Gateway or use the Core service as a routing proxy so that the front-ends only talk to the core, and core forwards requests to others (this can simplify client logic and centralize security). However, to **"keep it simple for now,"** we will start with minimal gateway logic: the Core service will handle login and maybe provide an entry-point API (like a service registry or user's available services), while the specific feature APIs (form, short link, etc.) will be accessed directly by their respective front-ends using the JWT for auth. This direct approach is straightforward, and we note that it can be refactored behind a unified gateway later without changing the fundamental microservice structure.

Authentication and Single Sign-On (JWT-based)

A critical part of the architecture is the **Single Sign-On (SSO) mechanism using JWT tokens**. Rather than each microservice managing user logins separately, the Core service will provide a unified authentication system:

- **Central Auth Service (within Core):** The core service will expose authentication endpoints (e.g., `/login`, `/logout`, `/register`, `/refresh-token` if implementing refresh tokens). When a user needs to log in (whether they came from the core portal or directly trying to access another service's UI), they will be redirected to the Core service's login page (the Auth UI). The user enters credentials (or potentially uses an OAuth provider in the future), and the core service verifies them (checking the database for username/password, etc.). On successful authentication, the core service will create a **JWT (JSON Web Token)** signed with a secret (or private key) that all other services trust.
- **JWT Token Contents:** The JWT will include **claims** such as the user's ID, username, their tenant (for multi-tenant mode), their roles/permissions, and also which services they are allowed to access (this could be a custom claim like `services`: `["form_builder", "short_link", ...]` determined by the admin). Because JWTs are signed and tamper-proof, other services can rely on this data to make

authorization decisions without needing to call back to the core service for each request ⁴. For example, if the JWT contains `tenant: acmeCorp` and `services: ["form_builder"]`, then the Short Link service (on receiving this token) can immediately know the user's tenant and see that "short_link" is not in the list, thus deny access (or the core might not even give them a link to short link UI in the first place).

- **SSO Flow:** Suppose a user wants to use the Form Builder app. If they are not already logged in, the Form Builder front-end will redirect them to the Core service's login page (perhaps with a URL parameter indicating to return to Form Builder after login). The user logs in at Core; Core issues a JWT (usually sent to the client). We can send the JWT in a secure cookie (with the parent domain) or simply redirect back to the Form Builder app with the token (e.g., as a URL fragment or stored in localStorage via a post-login script). After login, the user is sent back to the Form Builder front-end, which now has the token (e.g., retrieved from the cookie or redirect). The Form Builder front-end stores the JWT (likely in memory or local storage) and includes it in the `Authorization: Bearer <token>` header of each API request to its backend. The Form Builder service's Express middleware will verify the JWT signature and claims on each request. If valid, it attaches the user info (from token claims) to the request context, and the request proceeds. This way, **the user experiences a seamless login across services** – they log in once at the central place and then access any allowed service without logging in again, as long as the token is valid.
- **Stateless Auth:** We will use JWTs in a stateless manner – meaning the token itself is the proof of authentication and contains the needed info. We **do not need to store session state** in the backend for JWT auth (no need to look up a session for each request) ¹². Each service simply checks the token. This reduces cross-service chatter and central dependencies. One exception is if we want logout or token revocation; since JWTs are stateless, a simple logout on the client just removes the token. If we need to invalidate a token before it expires (say an admin revoked a user), we might use Redis to keep a blacklist of JWT IDs or a short token lifespan with refresh mechanism. Initially, we can issue JWTs with a reasonable short lifespan (e.g., 15 minutes) and issue **refresh tokens** (longer-lived, stored http-only cookie) if we want to avoid frequent re-login – but refresh token management could also be handled by the core service.
- **JWT Validation in Services:** All microservices will share the JWT verification logic. We can create a small shared middleware (or NPM package) for Express that knows how to verify our JWT signature (using a secret key or public key if using RSA) and perhaps enforce certain claims. For instance, an Express middleware for the Form Builder service might verify that `jwt.claims.services` includes `"form_builder"` before allowing access. Similarly, a tenant-aware service will check the `tenant` claim and use it to filter data queries (so a user from tenant "X" only sees tenant "X" data). This way, **authorization** is partly handled via JWT claims – the token itself tells if the user is allowed or not for a service, and what role they have. This approach aligns with common microservice security practice: a central IAM service issues tokens and the microservices trust those tokens to authenticate the user and identify their permissions ⁴.
- **Multi-Tenant Login:** In multi-tenant mode, a question is how the system knows which tenant the user is logging into. If each user account is tied to exactly one tenant, it's straightforward – the user's record in the database has a tenant ID. The JWT issued will include that tenant ID. If users can belong to multiple tenants, the login might need the tenant context (like "select organization"). But to keep it simple, we assume a user logs into one tenant context at a time (if needed, the core service login

UI could prompt for tenant identifier or subdomain etc., but that's an extension). In **single-tenant mode**, the tenant concept can be hidden entirely – all users belong to the only tenant, and the system might omit tenant in the JWT or use a default tenant value. We will provide a configuration flag (e.g., `MULTI_TENANT=true/false`) that the core service can use to either enforce tenant context or not. If multi-tenancy is off, the core service may treat all users as part of a single organization and perhaps elevate one user as "super admin" (the ultimate administrator) since there's no higher-level separation ¹³. If multi-tenancy is on, the core could have a "super admin" user who can create new tenants and manage system-wide settings, in addition to tenant-level admins.

- **SSO with JWT vs OAuth:** We are effectively building a custom SSO using JWT. This is suitable for a first version. In the future, one could integrate a full-fledged Identity Provider (OAuth2/OIDC based) or an IAM service (like Keycloak, Auth0, etc.) ¹⁴ ¹⁵. For now, the core service suffices. We ensure to follow security best practices: hash passwords in the database, use HTTPS in production so JWTs (if in cookie or header) are not intercepted, and implement proper CORS rules so tokens aren't leaked to untrusted origins.

Multi-Tenancy Considerations

Supporting both single-tenant and multi-tenant deployments means our design must be flexible. **Multi-tenancy** means a single running instance of the application serves multiple client organizations (tenants), isolating their data and configurations ⁷. We will incorporate multi-tenancy primarily at the data and configuration level:

- **Data Isolation Models:** We have options for how to separate tenant data:
- **Shared Database, Tenant ID column (Single Schema):** All tenants' data live in the same database and tables, but every table has a `tenant_id` field to partition the data. The application must **always filter by tenant_id** on queries to avoid leakage. This is the simplest to implement and will likely be our initial approach (especially if the number of tenants is not huge) ¹⁶. We must be careful with every query and enforce a global filter (we might implement a middleware that injects a `WHERE tenant_id = X` on queries, or use an ORM that can default-scope by tenant).
- **Shared Database, Schema-per-Tenant:** PostgreSQL allows multiple schemas in one database. We could create a separate schema for each tenant. All schemas have the same structure (same tables), but data is fully separated. The application chooses the schema based on the tenant context (some ORMs support dynamically selecting schema). This offers stronger isolation (one tenant's queries can't accidentally touch another's data, if configured right) and is moderately complex (managing migrations across schemas, etc.) ³. This could be a future optimization if needed.
- **Database per Tenant:** Each tenant gets their own database instance. This provides the highest isolation (both in terms of security and performance) ¹⁷ ¹⁸, but at the cost of significant overhead in managing potentially many databases (connections, migrations, backups, etc.) ¹⁹. This is usually only justified for large enterprise tenants or strict data isolation requirements. We will design the code to not preclude this option (e.g., by abstracting the data access by tenant), but we will likely not start with this model due to complexity.

For our use case, **a single database with tenant ID fields (option 1)** is a good start, as it keeps development simple and still allows multi-tenancy logically. We will implement safeguards to prevent cross-tenant access: for example, when a user makes a request, the service will determine the tenant (from the

JWT claim) and then ensure every database query includes that tenant condition (this could be manual, or using an ORM with tenant-scoped contexts). We might also maintain a mapping of tenants in the core service (like a `tenants` table listing each tenant and config, which is used by the super admin to enable/disable services per tenant, etc.).

- **Tenant-Specific Configurations:** The core service (or a config service) can hold settings that differ per tenant. For instance, one tenant might have the Form Builder service enabled, another might not. Or branding could differ by tenant (logos, themes). We can accommodate this by storing such settings in the database and having the front-ends adapt (e.g., when a user from tenant X logs in, the core service knows which services X has and can present links accordingly). Because the JWT might list allowed services for the user, that is directly tied to what their tenant (and their role) allows.
- **Single-Tenant Mode:** If configured for single-tenant, the application will essentially treat everything as one tenant. The UI might not even expose any tenant concept to the admin (there's just "the system" and its users). In this mode, the "super admin" is simply the admin of this sole tenant. We might implement this by setting a default tenant id (like `tenant_id = 1` for all data) or by not requiring a tenant filter at all. The multi-tenant code paths can be toggled off. This dual-mode capability should be documented and controlled by a configuration variable or build setting.
- **Role-Based Access Control (RBAC):** Alongside multi-tenancy, we should implement roles (e.g., normal user vs admin) and ensure they are scoped per tenant. For example, a user could be an "Admin" of tenant A (can manage users and settings in that tenant) but just a "User" in tenant B. Or in single-tenant mode, you either are an admin or user of the whole system. Roles will be reflected in JWT claims and enforced in services (e.g., only an admin token can access admin endpoints like creating new forms or managing short links globally). The Node.js boilerplate approach often integrates RBAC with multi-tenancy ¹³, and we will do similarly.
- **Tenant Onboarding:** In multi-tenant mode, when a new tenant signs up (for example, a new company customer), the super admin (or an automated process) would create a new tenant record. If using schema-per-tenant, we'd create a new schema at that time (this can be done programmatically or migration scripts). If using separate DB per tenant, we'd provision a new database. Initially, to keep it simple, we might manually configure tenants in the database. Eventually, a UI for "super admin" to add tenants can be part of the core service.
- **Testing Multi-tenancy:** We will need to test that data is properly isolated. For example, user from tenant A should not see or modify data of tenant B via any API. We might include the tenant ID in all relevant queries and maybe add automated tests that create data under two tenants and ensure no cross-bleed.

Front-End Structure and Shared Component Library

On the front-end side, we will implement a **micro-frontend-like structure**: separate Angular applications for each major service, plus possibly a main Angular application for the core portal. To ensure a consistent

user experience across these apps, we will create a **Shared UI Components Library** that all the Angular projects can use.

- **Multiple Angular Apps:** We anticipate at least three Angular projects to start:
 - Core (Home) App – handles the main landing page, login UI, and potentially a dashboard that links to other services.
 - Form Builder App – the UI for the form builder tool (accessible after login).
 - Short Link App – the UI for the URL shortener (if a UI is needed beyond maybe a simple form, but likely yes for managing your links).

Each of these will be a standalone Angular 20 application that can be deployed separately (perhaps at different paths or subdomains). For example, we might deploy Core at `https://app.myplatform.com/` and Form Builder at `https://forms.myplatform.com/` or a route like `/forms/`. In development, they run on different ports (e.g., core on localhost:4200, form builder on 4201, etc).

- **Shared Component Library:** We'll create an Angular library (using Angular CLI or Nx workspace) named something like `shared-ui` that contains common UI components and possibly services. This library will include things such as:
 - A common layout (navigation bar, side menu, footer) so that all apps have the same frame.
 - Reusable widgets (e.g., a custom text input, a date picker wrapper, etc.) possibly extending or styling PrimeNG components in a uniform way.
 - Styling guidelines via Tailwind CSS classes (we can configure Tailwind in the library so the same utility classes are available in all apps).
 - Possibly some utility services for front-end (like an authentication service to manage JWT and redirects, so we don't duplicate that logic).

By using a shared library, we avoid repeating code across Angular projects and ensure consistency. For example, if we update the top navigation bar (say to add a new service link), we do it in one place (the shared library) and all front-ends get the update.

- **Monorepo approach:** To simplify development, we can use a **monorepo** for front-end (and possibly back-end too). Tools like **Nx** (from Nrwl) allow managing multiple Angular applications and libraries in one workspace, as well as Node applications, with shared build config. We could set up an Nx workspace containing:
 - `apps/core-ui` (Angular app),
 - `apps/form-builder-ui`,
 - `apps/short-link-ui`,
 - `libs/shared-ui`,
 - and even Node apps like `apps/core-service` (Express backend) etc., in the same repo.

Nx will help with running them, sharing code, and even building Docker images, but adopting Nx might be a learning curve. Alternatively, we can manage a simpler structure: one Git repository for all front-end projects (Angular supports multiple projects in one `angular.json` workspace), and separate repos for each Node service. The **shared component library** could live in the front-end repo and be imported into

each Angular project via relative path or built and distributed via npm link. Either way, we ensure we have a **single source of truth for shared components**.

- **PrimeNG and Tailwind Integration:** We will use **PrimeNG** as a UI component suite for Angular to accelerate UI development (gives us ready components like tables, dialogs, charts). Tailwind CSS will provide utility-first styling to easily customize the look. We'll configure Angular to use Tailwind (which involves adding Tailwind to the build pipeline via PostCSS). PrimeNG components can be themed; we might use Tailwind utility classes in combination with PrimeNG where appropriate (or even look into something like PrimeNG Tailwind theme if available). The shared library can encapsulate the PrimeNG theme setup so all apps share the same theme (ensuring uniform styling). For example, if we decide on a particular color palette, we define it once.
- **Sharing Models and Types:** In addition to UI components, if we want, we can share TypeScript interfaces or client-side API services across apps. For instance, a type definition for `Form` or `User` can be placed in a shared library so both Core and Form Builder apps use the same interface, reducing mismatches. We might create a separate library for shared data models or utilities if needed (e.g., `libs/shared-models` with interfaces, or even an autogenerated API client from OpenAPI definitions of our backends). This is optional but beneficial for consistency.
- **Routing and Navigation:** Each Angular app will handle its own routing internally for its features. The Core app will have routes for home, login, signup, admin panel, etc. The Form Builder app will have routes like `/forms`, `/forms/new`, `/forms/:id/edit`, etc. From the user's perspective, navigating between micro-frontends should be seamless. We might implement a unified navigation menu that spans apps – one approach is to have the Core application serve as a shell that can load others (micro-frontend architecture using module federation), but that can be complex. Simpler: we have a top nav bar (from shared library) that has links to the other apps (maybe as normal hyperlinks). Clicking "Form Builder" could simply redirect the browser to the Form Builder app's URL. Because we have SSO, the user will already be logged in, so the Form Builder app will detect the token and not prompt login again. There might be a slight page reload, but that's acceptable initially. Alternatively, if we invest in micro-frontend integration, we could dynamically load the other app into the shell, but let's keep that as a future enhancement.
- **Authentication on Front-end:** We'll implement an **Auth guard** in each Angular app. The guard will check if the JWT token is present (and possibly valid/not expired) before allowing access to protected routes. If not, it will redirect the user to the core login page. Also, each app might have an interceptor to attach the JWT in API requests. The shared library can provide these building blocks (like an AuthService that any app can use to get current token, login, logout, etc.). This way, all front-ends handle auth in a consistent manner.
- **Multi-Tenant UI:** If multi-tenancy is on, the UI will reflect that. For example, after login, the user's tenant name might be shown (e.g., "Acme Corp Dashboard"). If the user is a super admin (managing multiple tenants), the core app might have pages to manage tenants or switch tenant context. We can incorporate tenant-specific branding (like logo) by loading it from the tenant config. These details can be incrementally added once basic functionality is in place.

Data Management (PostgreSQL & ORMs)

Each microservice will manage its own data storage using PostgreSQL. We will likely run a single PostgreSQL server with multiple databases or schemas for simplicity, but logically treat them as separate databases for each service:

- **Core Service Database:** Stores users, credentials, roles, tenant info, and service access info. Key tables might include `users`, `tenants` (if multi-tenant enabled), `user_tenant_roles` (mapping which users belong to which tenant with what role), and perhaps `services` (list of services/modules and which tenants have them enabled, or which users have access). In single-tenant mode, many of these tables reduce to trivial cases (one tenant). This database is central to authentication and authorization decisions.
- **Form Builder Database:** Stores data like `forms` (form definitions), `form_fields` (questions), and `form_responses` (submitted answers). All records should have a `tenant_id` (unless single-tenant) to separate orgs. Possibly, it might reference a `user_id` from core for ownership (we might use a foreign key or just store the user ID as a string, since core's user might be in a different DB – we could use UUIDs for user IDs to identify globally). We could also keep an internal mapping of core user ID to its local representation if needed, but likely not needed if we trust core user IDs from JWT.
- **Short Link Database:** Stores `links` with fields like short code, target URL, creation user, timestamps, maybe usage count, etc. If multi-tenant, include `tenant_id` so each tenant's links are separate (or possibly each user only sees their own links, but admins could see all of their tenant's links). Could also have a `user_id` field referencing core users (for audit of who created the link).
- **Database Design Principles:** Each microservice can use its own ORM or query approach to interact with Postgres. For Node, popular ORMs are **Prisma**, **TypeORM**, **Sequelize**, or using query builders like **Knex**. The multi-tenant boilerplate suggests using Prisma for multi-tenancy with Postgres ⁶, which is a good modern option. Prisma with PostgreSQL can be set up to use schema-based multitenancy or even separate DB strings per tenant. Alternatively, using a lower-level approach (like raw queries with pg library or Knex) might give us more direct control for multi-tenancy. We'll choose a consistent approach for all services. For maintainability, an ORM like Prisma or TypeORM can be helpful (they can manage migrations and provide a consistent modeling of data).
- **Ensuring Data Isolation:** If using the shared schema approach, we will implement a convention (like always call a function to get the repository/model which applies tenant filter). If using Prisma, we might configure a middleware in Prisma client to always add `where tenant_id = X`. If using TypeORM, perhaps use entity scopes or simply be disciplined in queries. We may also use database constraints to prevent cross-tenant references (though if all tenants share tables, hard to enforce via constraints beyond unique composite keys like unique (tenant_id, something) if needed).
- **Transactions and Saga:** For inter-service operations, each service will handle its own transactions. Distributed transactions (across multiple DBs) are not in scope (we will rely on eventual consistency via events if needed). For example, if an action in Form Builder triggers something in Short Link, we won't try a single ACID transaction; instead, we do eventual via Kafka events and compensating actions if needed.

- **Caching with Redis:** Using Redis as a cache can significantly reduce database load. We'll identify areas to cache:
 - Static reference data (if any) – e.g., if we had a list of countries or large form templates that are read often.
 - Frequently accessed dynamic data – e.g., for Short Link, once a short code is created, store its target in Redis. Then the redirect operation (which needs to be very fast) can first check Redis. If found, no need to hit Postgres. If not, query Postgres and then populate Redis for next time. We would also set an expiration on these caches to auto-invalidate.
 - User session info – though JWT doesn't need server session, if we implement a refresh token or want to quickly check if a token is revoked, we might store a token ID in Redis with TTL equal to token expiration to allow logout invalidation.
 - Rate limiting – we can use Redis to count requests per IP or user in a time window for each service to mitigate abuse (e.g., more than X requests per second triggers a throttle).

Redis will be run as a separate service (likely one instance that all microservices can connect to). Because it is in-memory, it's extremely fast and suitable for ephemeral data. We must handle cache invalidation properly (e.g., if data changes in DB, we should update or invalidate relevant cache entries; we might not need very heavy caching at first, but good to design the option in).

Messaging with Kafka for Asynchronous Work

Integrating **Apache Kafka** allows services to communicate asynchronously and handle background processing robustly. We will set up a Kafka cluster (for dev, a single Kafka broker is fine; in production, likely a small cluster for reliability). Our services will use a Kafka client library (like `kafka.js` or `node-rdkafka`) to produce and consume messages.

Use cases for Kafka in our system: - **Event Broadcasting:** When certain events occur in one service, publish an event so other services can react. For example: - User registration (Core) -> event "user.registered" -> Form Builder service might consume it to create a default welcome form, Short Link service might not care, an Email service (if exists) would consume to send welcome email. - Form submitted (Form Builder) -> event "form.submitted" -> maybe an Analytics service would consume to log metrics, or an Email service to notify someone. - Short link clicked (Short Link) -> event "link.clicked" -> Analytics could record, or maybe trigger a notification if certain thresholds. - **Decoupling heavy processing:** If a request in a service requires heavy work, the service can offload via Kafka. E.g., user uploads a large file in Form Builder -> service stores it and emits "file.uploaded" -> another service or a background worker consumes and processes the file (like virus scan or generating preview). This way the HTTP request can return quickly and the rest happens asynchronously. - **Consistency between services:** Kafka can be part of ensuring eventual consistency. If the Core service is the source of truth for user data, it could emit events on user updates and other services might update their local data or caches accordingly. - **Central Event Logging:** We can also use Kafka as an audit log pipeline – all services could produce standardized events for significant actions, and a logging service (or just a Kafka consumer that writes to a log store) can persist those for debugging/compliance.

Implementation details: - We'll define **Kafka topics** for each event type or domain. Possibly use a convention like `{service}.{event}`. The Medium example used topics "user-created", "order-created" etc ²⁰. We can maintain a central list of topics (maybe in code or config) to avoid typos ²¹. - Each service will have a Kafka producer instance to send events, and a consumer instance to listen for events it cares

about. For instance, the Form Builder service might produce `form.submitted` events and consume `user.registered` events. - We'll keep event payloads small, typically just IDs and minimal info, and if more data needed the consuming service can fetch from source via API if necessary. Or include necessary info in the event to avoid coupling (e.g., include user email in the `user.registered` event so Email service doesn't have to call Core to get it). - Ensure idempotence on consumers: since Kafka may deliver events more than once in some failure cases, consumers should handle duplicates gracefully (e.g., by checking if they've processed an event ID already). This might not be critical for our initial stage but we keep in mind for design.

By leveraging Kafka, the architecture becomes more scalable: services remain **loosely coupled** – they don't call each other directly for most operations, reducing dependencies and allowing each to scale or fail independently ⁵. For example, if the Email/Notification service is down, users can still register and use forms; the "user.registered" events will accumulate in Kafka and when the email service comes back up, it will process the backlog and send emails.

Deployment, Containerization, and CI/CD

We aim to make the system **production-ready** with containerization and CI/CD from the start:

- **Docker Containers:** Each microservice (backend) will have its own Dockerfile. Typically, it will:
 - Use a Node.js base image (e.g., `node:18-alpine` for slim image).
 - Copy the service code into the image.
 - Install dependencies (`npm install --production` or similar).
 - Set environment variables (for DB connection strings, service-specific config, secrets like JWT signing key, etc. – these will be provided at runtime via env vars).
 - Expose the service's port (each service likely listens on a different port, e.g., 3000 for core, 3001 for form, etc., but in Docker environment the actual port mapping will be handled).
 - The container, when run, will start the Node.js service (e.g., `CMD ["node", "dist/server.js"]` or using a process manager if needed).

For Angular front-ends, we have a couple options: - We can build the Angular app to static files (HTML/JS/CSS) and serve them with a simple web server (like nginx or even the Express backend of the service). For example, the Form Builder UI could be deployed by hosting its files on an S3 or CDN, but to keep everything in sync, we might containerize the front-end as well using an nginx image. A Dockerfile could use `node:18` to build (`ng build`) then `nginx:alpine` to serve the `dist` folder. - Alternatively, each Angular app could be integrated with its corresponding backend to simplify deployment (for instance, the Form Builder service's Express app could also serve the Angular app's files). However, separating front-end and back-end into different containers gives more flexibility in scaling them independently or updating front-end without touching back-end.

We'll likely containerize front-ends separately. That means at runtime, an `nginx` container serves the static files for each Angular app. We'll need to configure it to route API calls to the appropriate backend or

use relative paths if the front-end domain differs (CORS). Another approach: serve front-end on one domain and APIs on subdomains or paths.

- **Docker Compose & Development:** Although the user prefers not to use Docker for local development, we can still provide a `docker-compose.yml` for convenient deployment in other environments or for a quick test of the whole system. For example, a docker-compose might define:
 - `core-service` (build from core Dockerfile),
 - `form-service`,
 - `shortlink-service`,
 - `core-ui` (nginx serving core Angular),
 - `form-ui`,
 - `shortlink-ui`,
 - `postgres` (with maybe multiple databases or just one if using schemas),
 - `kafka` (along with a `zookeeper` if needed, or using a pre-built image like `bitnami/kafka` that can run without separate Zookeeper since newer Kafka can be started in KRaft mode),
 - `redis`.

This would allow someone to spin up the entire stack with one command (useful for testing the integration or running in a staging environment). In production, we might not use docker-compose but rather a Kubernetes YAML or Helm charts for more robust orchestration.

- **Environment Configuration:** We'll store config like DB connection strings, Kafka broker addresses, JWT secrets, etc., in environment variables or config files. For example, in Docker/K8s, the Core service might get `DATABASE_URL`, `JWT_SECRET`, `KAFKA_BROKER`, etc., injected. We should make sure secrets (like JWT signing key) are not baked into images but provided at runtime.

- **CI/CD Pipeline:** We will set up continuous integration and deployment such that:

- On each commit or pull request, automated tests (unit tests for services and perhaps front-end tests) are run. We aim to maintain good test coverage for critical pieces (like auth).
- On merging to the main branch (or on a version tag), the pipeline will build Docker images for each service and push them to a container registry (like Docker Hub or AWS ECR).
- The pipeline can then trigger a deployment. If using a simple VPS, this could be a script pulling the new images and restarting containers (maybe via docker-compose pull). If using Kubernetes, we might update the image tags and let the cluster roll out new pods. We might incorporate a tool like **GitHub Actions** for CI and perhaps **Terraform/Ansible** for managing deployment infra if needed. The key point is to automate as much as possible so that code changes go to production with minimal manual steps.

- **Local Development:** In local dev mode, each developer can run:

- PostgreSQL locally (or via Docker if they choose; we can provide a Docker command to run Postgres and Kafka if they don't want to install them).
- Kafka and Redis locally (Kafka can be a bit heavy to run; for dev, using Docker for Kafka might be easiest, or use a lighter alternative like in-memory message bus if absolutely needed; but let's assume dev uses Docker for Kafka/Redis or a shared dev instance).

- Run each Node service on their machine (`npm run dev` which might use nodemon for auto-reload).
- Run each Angular app with `ng serve`. We'll have to configure the Angular dev server proxy or CORS: e.g., when developing Form Builder UI at localhost:4201, and Form Builder API at localhost: 3001, we either enable CORS on API or set up proxy so that calls to `/api/form/*` on Angular dev are forwarded to 3001.
- This non-docker local workflow gives fast refresh and debugging capabilities (you can attach a debugger to Node processes, etc.). We ensure that our code can read config from a `.env` file or similar when not in Docker.
- **Deployment strategy:** For production, containerization ensures the app can run anywhere. We can deploy on a cloud VM with docker-compose, or on a Kubernetes cluster for scalability. The architecture, being microservices, inherently supports scaling out each service independently (for instance, if Form Builder becomes heavily used, we can run multiple instances of it behind a load balancer, and similarly for others). We should include a **load balancer** in front of each service (or an ingress controller if K8s) – often the API Gateway serves this role, but if we go with direct calls, we need either separate subdomains or URL paths for each service. A likely approach is to have a single domain and an Nginx or API Gateway routing based on path: e.g., `/form-builder-api/**` goes to Form Builder service, `/shortlink-api/**` goes to Short Link service, and `/core-api/**` goes to Core service. The Angular apps could be served at different subpaths or subdomains accordingly. This routing could be handled by a containerized Nginx in the docker-compose or by cloud infrastructure. We won't detail too much here but will ensure the blueprint acknowledges this requirement.
- **Monitoring and Logging:** (Not explicitly asked, but for completeness) In a microservice system, it's important to have centralized logging (maybe use something like ELK stack or a cloud logging service) and monitoring (Prometheus/Grafana, etc.). For now, we'd at least ensure each service logs to console (Docker will capture those) and maybe structure logs with context (including tenant and user IDs for traceability). If resources allow, we integrate simple health checks and metrics.

Now, let's outline a **step-by-step roadmap** to implement this system from scratch:

Implementation Roadmap

To build this project in stages, we will follow a structured approach:

1. Project Scaffolding and Repository Setup:

- Set up a version control repository (e.g., Git) for the project. We can choose a monorepo structure to house all services and apps together (using Nx or Lerna), or use separate repos per service. For simplicity, let's assume a single repository containing multiple folders: e.g., `core-service/`, `form-service/`, `shortlink-service/`, `shared-lib/`, `core-ui/`, `form-ui/`, `shortlink-ui/`.
- Initialize each Node.js service with its own `package.json` and base Express application. A common structure for Express projects will be used (controllers, routes, models, etc.) ²² ²³ for

consistency. For example, create a simple Express server in `core-service/app.js` that listens on a port and has a health check route.

- Initialize the Angular workspace (using Angular CLI) for multiple projects. We run `ng new platform-web --create-application=false` to create an empty workspace, then generate applications: `ng generate application core-ui`, `ng generate application form-builder-ui`, `ng generate application short-link-ui`. Also, generate a library: `ng generate library shared-ui`. Set up Tailwind CSS in the workspace (install tailwind, create `tailwind.config.js` and include it in Angular styles).
- Add PrimeNG to the Angular projects (install primeNG and a theme, e.g., `npm install primeng @angular/cdk @angular/animations`). We'll integrate PrimeNG components in the shared library (for example, create a SharedModule that imports PrimeNG modules and exports common components).

2. Core Service – Authentication & User Management (Backend):

- Design the database schema for core: users, tenants, roles, etc. Write SQL migration or use an ORM (e.g., if using Prisma, define the schema.prisma with User, Tenant, etc. If using TypeORM, define entities).
- Implement the User model with fields: id (UUID), username/email, password hash, tenant_id (nullable or fixed if single-tenant), roles, enabled services etc. Implement a Tenant model if multi-tenant (with tenant id and name).
- Set up Express routes for `/auth`:
 - `POST /auth/login` to authenticate (check username/password against DB, using bcrypt to verify hash) – on success, generate JWT (using `jsonwebtoken` library) and send it back (as JSON response containing the token, and perhaps set as HttpOnly cookie).
 - `POST /auth/logout` (optional, mostly client-side token removal, but could handle blacklist).
 - `POST /auth/register` if self-service signup is allowed (optional; or maybe only admins create users).
 - `GET /auth/profile` as a test endpoint to return current user info if JWT is valid (protected).
- Implement JWT generation and signing. The secret key should be stored in an env var (e.g., `JWT_SECRET`). Decide on token expiration (e.g., 15m or 1h). Also consider including a `refreshToken` issuance: e.g., on login, along with access JWT, give a refresh token (httpOnly cookie) that can be used to get a new JWT when the old expires.
- Add middleware for protected routes to verify JWT. For example, use `express-jwt` or custom middleware that reads `Authorization` header, verifies the token signature and then attaches `req.user` with token claims.
- Test the login flow using a REST client (simulate a user logging in and calling a protected endpoint).
- Implement user and tenant management endpoints: e.g., `POST /tenants` to create a new tenant (only core super admin can call), `POST /users` to create a user (maybe by admin), and some GET endpoints to list users (with tenant filtering).
- Ensure that in multi-tenant mode, these endpoints respect tenant context (e.g., an admin of a tenant can create user only in their tenant, etc.).
- **SSO logic:** Determine how the Core service will handle login redirects. Possibly, if a request comes with a query like `?redirectTo=form`, after successful login we respond with a small HTML/JS that redirects the browser to the Form Builder app URL with the JWT attached. Alternatively, if using

cookies for JWT, and the Form Builder app is on the same parent domain, the Form Builder can simply detect the cookie.

- For initial simplicity, we can skip the automated redirect and assume the user will click "Go to Form Builder" after logging in. But ideally, to meet the requirement, implement a parameter: e.g., Form Builder sends user to `https://core.myapp.com/login?service=form-builder`. After login, core redirects to `https://forms.myapp.com?token=...`. The Form app picks up the token from URL and stores it.
- The core service, at this point, can also serve as an index of services. Create an endpoint like `GET /services` that returns the list of service URLs the user has access to (based on their token or roles). The core UI will call this to display links.

3. Core Front-End (Angular) – Home & Login UI:

- Develop the Core Angular app (`core-ui`). Create components for Login, Register (if needed), Home Dashboard.
- The Login component will have a form bound to username & password, and on submit it calls the core service's `POST /auth/login` (using `HttpClient`). On success (token returned), store the token (e.g., in `localStorage` or in a cookie if backend set one). If there's a redirect target (we might pass it through a query param or a shared service), handle that: e.g., use `window.location.href = targetUrl` to send user to the other app along with the token if needed.
- For Home Dashboard, if user is logged in, display some welcome message and a menu of available services. This page will call `GET /services` (as mentioned) to know what services to show. For each service (like Form Builder), show a card or link. When clicked, it navigates to the service app (e.g., opens `forms.myapp.com`). Because SSO is in place, the user should already be authenticated there (the token travels via cookie or the service will redirect them back if missing).
- Use the shared UI library for styling: e.g., have a common `<Navbar>` component that shows the app name and maybe a logout button. On logout button click, clear token and redirect to login.
- Apply route guards: for protected routes (like `/dashboard`), if no token present, redirect to `/login`. The guard can also optionally verify token expiry and trigger refresh or re-login.
- Test the core UI: run it with the core backend. For development, set up a proxy so that Angular dev calls `/auth/login` goes to `localhost:3000` where core service runs. Verify you can register a user (if implemented) and log in, get to home page.

4. Shared UI Library Development:

- Flesh out the `shared-ui` library with common components. For example, create:
 - `HeaderComponent` with a top bar (maybe includes the product name, and if user logged in, their name and a logout link).
 - `FooterComponent`.
 - `AuthGuard` service (not a visual component but an Angular service/guard to protect routes, reading from a common `AuthService`).
 - `AuthService` that is responsible for storing JWT, providing methods to login (calls backend) and logout, and a helper to check permissions (like `hasServiceAccess(serviceName)` reading from token claims).
 - Styles: maybe in `styles.scss` of shared we define Tailwind utilities or include Tailwind base styles. Also if we want to define some CSS variables for theme (color palette to match Tailwind config).

- The library should be setup in `tsconfig` so that each app can import from `@shared-ui` (depending on our build setup, this might be automatic if using Nx).
- Use PrimeNG in the library: for instance, create a `ButtonComponent` that wraps `p-button` with our custom styling, or simply re-export PrimeNG modules. Possibly create a theme file for PrimeNG (could use Tailwind to override PrimeNG CSS, or we pick a PrimeNG theme).
- Build the library and ensure the apps compile with it. Replace any duplicate code in core-ui with library components.

5. Form Builder Service (Backend) Development:

- Design the database schema for forms. Tables: `forms` (id, name, description, created_by, tenant_id, etc.), `form_fields` (id, form_id, field_type, label, etc.), `form_responses` (id, form_id, submitted_by, submitted_at, maybe a JSON or a related table for answers).
- Implement the Express app in `form-service`:
 - Set up JWT auth middleware (reuse from core or use a package) to protect all endpoints (except maybe a future `/public/{formId}` endpoint if forms are to be publicly accessible; but assume it's for internal use by logged users).
 - Define routes:
 - `GET /forms` (list forms the user has access to – likely filter by `tenant_id` from token, and possibly by user if needed).
 - `POST /forms` (create a new form – uses token info for created_by and tenant).
 - `GET /forms/:id` (get form details, ensure it belongs to same tenant).
 - `PUT /forms/:id` (update form).
 - `DELETE /forms/:id`.
 - `GET /forms/:id/responses` (list submissions).
 - `POST /forms/:id/responses` (submit a new response – this might be used if, say, filling out a form in future; but if the form builder is more for designing forms, maybe responses come through a different channel).
 - Ensure to enforce multi-tenancy in queries (e.g., `WHERE tenant_id = req.user.tenant`).
 - Integrate Kafka: e.g., after a new form is created, produce an event "form.created". After a response is submitted, produce "form.response.created". These could be used by other services or future analytics.
 - (Optional) If heavy operations needed (like generating a PDF of responses), push to Kafka instead of doing synchronously.
 - Use Redis caching if needed: e.g., cache form definitions to speed up loading for frequent access.
 - Testing: Use an HTTP client to hit these endpoints with a JWT obtained from core login. Confirm that without JWT you get 401, with JWT you get correct data.

6. Form Builder Front-End Development:

- Build the Angular app for the form builder (`form-builder-ui`):
 - On startup (AppComponent init or a guard on the main route), check for JWT via AuthService. If not present, redirect to core login. Possibly the form app could accept a `?token=` param if coming from a redirect and use that to set AuthService.
 - Create components:
 - Form list component (shows list of forms, "Create New Form" button).

- Form editor component (to design a form – could be complex, but maybe start simple with a form title and some fields).
- If implementing responding via this app, also a component to view responses.
- Use the shared Header component so that the user sees a common nav (with maybe a link back to home or other services).
- Use PrimeNG components for building the UI (e.g., dropdowns for field types, drag-and-drop ordering maybe – though that can be complex; initially can just add fields in a list).
- Integrate with back-end via HttpClient:
 - Call `GET /forms` to display list.
 - Call `POST /forms` to create a new form.
 - etc., hooking these to UI actions.
- Ensure to include the JWT in calls. We can use an `HttpInterceptor` from shared that reads token and sets `Authorization` header.
- Test by running the form app (ng serve) and core service. Try to access form app without logging in – it should redirect to core or show a message and link to login.
- After logging in on core, manually go to form app (or if we implemented redirect, it happens automatically) and verify data loads properly.

7. Short Link Service (Backend) Development:

- Schema design: `links` table (id or short code as primary key, full URL, created_by, tenant_id, created_at, maybe a counter of clicks). Possibly a separate table for click events if tracking each access.
- Express routes:
 - `POST /links` – create a new short link (generate a short code, ensure it's unique, save with tenant and user).
 - `GET /links` – list all short links (for this tenant or user).
 - `GET /1/:code` – this is the redirect endpoint for when someone uses the short URL. This one is public in the sense that anyone with the link can hit it, so it should not require JWT (we cannot expect end-users to be logged in to follow a short link). It will find the target URL by code and redirect (HTTP 302) to it. For security, if we want to restrict short link usage to within tenant, that gets tricky for public links; usually short links are open. We'll assume open access for redirect.
 - `DELETE /links/:code` – to delete a short link (auth required, only creator or admin).
 - Optionally, `GET /links/:code/stats` – to get click stats (protected).
- Use JWT middleware on all endpoints except the public redirect. For the redirect, we must ensure at least that if the code exists but belongs to a private tenant, it may still be accessible publicly. We'll treat short links as globally accessible for simplicity (like bit.ly).
- Implement logic to generate short codes (could be a hashid or just a random string). Check collisions.
- Use Redis caching: this is a prime candidate. When `GET /1/:code` is called, first check Redis cache for that code's target. If found, immediate redirect. If not, query Postgres, get URL, store in Redis (with a TTL of say 24h), then redirect. This cache will drastically reduce DB hits on popular links.
- Kafka integration: produce an event "link.clicked" each time a redirect happens (could carry link id, timestamp, maybe source IP if not sensitive). Also "link.created" when a new link is made. Other services (like an Analytics service or core) could listen and accumulate usage data or trigger notifications.

- Test creating and using a short link via API (create one with JWT, then simulate a user clicking by calling the redirect URL without JWT).

8. Short Link Front-End Development:

- Build `short-link-ui` Angular app:
 - Similar auth guard setup as others.
 - Components:
 - Short Link list (show all links user created, with stats maybe).
 - New Link form (input for original URL, maybe an option to customize code).
 - Possibly a component to show details of a link (with a list of clicks if we store them).
 - Integrate with back-end:
 - Call `GET /links` to populate list.
 - Call `POST /links` to create new.
 - Show the short URL to the user (the domain plus code).
 - If we track stats, call some endpoint to get click counts.
 - Use shared components for header etc.
 - Test the UI in integration with the service.

9. Integration of Services and SSO Flow:

- At this point, ensure the end-to-end SSO works:
 - Start core-service, form-service, shortlink-service, and the Angular apps.
 - Try accessing Form Builder app directly: should redirect you to login (core UI).
 - Log in at core: after login, manually go to Form Builder app (or if we wired redirect, it goes automatically). You should now see the form builder UI without another login. If not, debug token storage/transfer.
 - Similarly test going to Short Link app.
 - Test logging out: clicking logout on one app should ideally log out from all (since it's stateless, just removing token is enough – but if using cookies, ensure cookie cleared). Probably implement logout such that it clears token in localStorage and also hits core `/logout` (which might do nothing but for completeness).
- Multi-tenant test: create two tenants in core (if possible manually via DB or an API). Create users for each. Ensure a user from tenant A cannot access tenant B's data by calling APIs with their token (should get empty or 403). This tests our tenant enforcement.
- Adjust any configurations: e.g., make sure services know the Kafka broker address via config, and that Kafka topics are created (some Kafka clients auto-create topics on first publish, or we might pre-create them).
- Logging: ensure each service logs meaningful messages. Possibly include request logging in dev.

10. Kafka and Background Services:

- Stand up a Kafka instance (for dev, use docker). Configure services to connect (e.g., Kafka at `localhost:9092`).
- Implement consumer logic in services:
 - Core might not need to consume much, but maybe it could listen for events like "form.response.created" to, say, increment a counter on the user's profile, etc.

- Form Builder could consume "user.registered" to auto-create a welcome form (just an example).
- Short Link could consume nothing initially.
- If we had an Email service, it would consume "user.registered", "form.response.created", "link.clicked" etc. (We might not implement it now, but we leave placeholders).
- Ensure each consumer runs in a separate thread or is non-blocking (Node is single-threaded but Kafka client is async). Use group IDs for consumers so if we scale services they share load.
- Test producing events: after a user registers, check Kafka (maybe using a CLI tool or a simple temporary consumer code) to see the event. Or simpler, log in each service's consumer callback to confirm reception.

11. Redis Integration:

- Start a Redis server (again likely via docker for dev).
- Integrate Redis in Short Link service for caching: use `redis` or `ioredis` npm package. Test that after first DB lookup, subsequent redirects are served from cache (maybe log when DB hit vs cache hit).
- Integrate Redis in core (optional): could use it to store active sessions or rate-limit login attempts (to prevent brute force, e.g., using a key for login attempts by IP).
- Integrate in form if needed (maybe caching form schema).
- Make sure to handle cache invalidation: e.g., if a short link is deleted, purge it from Redis as well.
- Test the performance or at least correctness with Redis on and off.

12. Containerization Setup:

- Write Dockerfiles for each Node service:
 - Use multi-stage build if we want to minimize image size (first stage: build if we transpile TS to JS, second stage: just Node to run).
 - Ensure to `COPY package.json` and `npm install`, then copy code.
 - Set `CMD ["node", "app.js"]` or whatever the entry is.
- Write Dockerfiles for Angular apps:
 - Stage 1: use `node:current-alpine` to `npm install` and `ng build --prod`.
 - Stage 2: use `nginx:alpine`, copy the `dist` output to `/usr/share/nginx/html`.
 - Add an nginx config if needed to handle any routing (like using HTML5History, need to redirect 404s to index.html; we can include a default config for that).
- Create a `docker-compose.yml` for all services:
 - Define each service container, link them in a network.
 - Use environment variables for config (for DB connection, we might link them to a single Postgres container, but if each service expects its own DB, we can use one Postgres and multiple databases or schemas).
 - Add Kafka container (maybe use `wurstmeister/kafka` or bitnami images) and a Zookeeper if needed (or a new KRaft mode Kafka if simpler).
 - Add Redis container.
 - Set up dependencies ordering (docker-compose can ensure DB is up before services, etc., by using `depends_on`, though for Kafka might need a wait script).
- Test `docker-compose up` locally: all containers build and run. Then try to access the system through the core UI container. This will surface any missing config or networking issues. Adjust as

needed (maybe enabling CORS if front-ends are on different domains/ports; in docker, they might all be on same domain under different paths if we set up nginx reverse proxy).

- Possibly set up an **NGINX gateway container** in the compose: For instance, an Nginx that listens on port 80 and routes:
 - `/` to core-ui container,
 - `/core-api/` to core-service,
 - `/forms/` to form-ui,
 - `/form-api/` to form-service,
 - `/links/` to shortlink-ui,
 - `/shortlink-api/` to shortlink-service,
 - This way from a user's perspective, it's one host. This is optional for the compose, but likely for production we'd do something like that or use subdomains.
- Document how to run everything with Docker and how to run in dev mode without Docker.

13. CI/CD and Deployment Prep:

- Set up a GitHub Actions workflow (or other CI tool) in the repo:
 - Steps: checkout code, set up Node (and maybe docker if building images), run `npm install` in each service and run tests, run `ng build` in each Angular and maybe run Angular unit tests.
 - If all tests pass, build Docker images. We can use Docker buildx to build multiple images (one for each service and UI). Tag them with commit hash or a version.
 - Push images to a registry.
 - Deploy: If we have a test server, we could have the CI ssh into it and run docker-compose pull & up. Or if K8s, update manifests via `kubectl` or helm upgrade.
 - Ensure secrets (like JWT secret, DB passwords) are managed via CI secrets and not stored in code.
- Not all these need to be fully implemented now, but outline the process and maybe implement a basic CI for building and testing to catch issues early.

14. Extending & Scaling Considerations:

- Document how a new service can be added (detailed in next section).
- Consider performance tuning: use of load balancers for services, scaling out Kafka consumers if needed (like partition topics so multiple instances can share the load).
- Monitoring: integrate something like pm2 for process management in Node (with clustering if one machine runs multiple threads of same service).
- Security audit: ensure all external inputs are validated, use HTTPS, set appropriate CORS (the front-end domains should be allowed in CORS or using same-site cookies for JWT).
- Prepare for production: e.g., set up database migrations properly, so we can apply schema changes without downtime (maybe use a tool like Flyway or the ORM migration).

With the above roadmap, the team can develop the system iteratively, verifying each piece before moving to the next. Starting with core auth ensures the foundation (SSO) is solid, then building each service on top.

Extending the Architecture: Adding New Services

One of the benefits of a microservice architecture is the ease of adding new independent services. Suppose later we want to add a new feature (e.g., a **Survey Service** or **Notification Service**). Here is how we would integrate a new service into the existing structure:

1. **Scaffold the New Service (Backend):** Create a new Node.js Express app for the service (either in the monorepo or its own repository). Follow the same project structure conventions as existing services for consistency (routes, controllers, models, etc.) ²². Set it up with its own database tables (or schema) relevant to its domain. For example, a Survey service might have `surveys` and `responses` tables. Implement the core logic and endpoints for this service.
2. **Integrate Authentication:** Import or copy the JWT auth middleware into the new service. Configure it with the same JWT secret/public key so it can validate tokens issued by the core service. Protect all endpoints that should be secured. Define what roles or claims are needed – for instance, ensure the JWT's allowed `services` claim includes this new service's identifier for any user trying to use it. You might need to update the core service to include this service in the list when assigning access to users/tenants. For example, add a new service identifier "survey" in the core database and when an admin enables it for a tenant or user, the JWT issuance includes "survey" in the services claim.
3. **Multi-Tenancy & Data Isolation:** If multi-tenancy is used, implement tenant awareness in the new service. This means:
 4. Include a `tenant_id` column on its data tables.
 5. When handling requests, get the tenant from `req.user.tenant` (from JWT) and use it to filter data queries and to assign to new records.
 6. If using separate schemas or DBs per tenant approach, set up the new service to manage connections accordingly (similar to how other services do). For example, if we had a function to get a tenant-specific DB connection ²⁴, reuse that pattern.
7. **Front-end for the New Service:** If the service is user-facing, generate a new Angular app for it (similar to form-ui and shortlink-ui). Use the shared component library to maintain consistent UI. Implement the UI screens needed for that service. For instance, a Survey service UI might let users create surveys and view responses. Make sure to add the authentication guard as with other apps, so it redirects to login if no token. Configure the Angular app to call the new service's API (perhaps via a proxy or direct calls to its domain). Also, add a link in the Core app's dashboard to this new service (e.g., if user has access, show "Go to Survey" button linking to the new app's URL).
8. **Update Core Service (if needed):** The core service might need slight modifications:
 9. If we maintain a list of services in the database, add the new service there.
 10. Possibly add admin UI in core to toggle this service per tenant (unless by default everyone gets it).
 11. Ensure user roles or permissions can include the new service. For example, when creating a user, the admin can mark them as having access to Survey service.
 12. Update JWT generation logic to include the new service in the `services` claim for relevant users.

13. **Messaging Integration:** Decide if the new service should produce or consume Kafka events:
 14. If it needs to react to events (e.g., a Notification service might consume "form.response.created" to send an email), set up Kafka consumer logic in it.
 15. If it produces events (e.g., Survey service might emit "survey.published" or "survey.response.created"), define those topics and produce accordingly. Update the centralized event constants (if we have one) to include new event keys ²⁵.
 16. Other services can be left untouched unless they need to respond to these new events. For instance, core service might not care about survey events, so no change there.
17. **Redis/Caching:** If the new service can benefit from caching, utilize Redis. Perhaps none needed initially; but if something like caching survey results, it's available.
18. **DevOps:**
 19. Add the new service to Docker Compose (new service container, new front-end container if applicable).
 20. Create Dockerfile(s) for it (backend and frontend).
 21. Adjust CI/CD pipelines: include building and testing the new components. Ensure the new images are built and pushed.
 22. If using an Nginx gateway, update its config to route to the new service/UI.
 23. Add environment variables for the new service in compose or K8s manifests (DB connection, etc.).
 24. Ensure secrets (like any new JWT secret if we had separate, but we likely use the same one) are handled.
25. **Testing the New Service Integration:** Try logging in as a user who should have access to the new service and navigate to it. The SSO should allow them in without a separate login. Test a user without access – they should either not see the link or if they somehow navigate there, the service should reject their JWT (we can implement that check by reading the `services` claim and returning 403 if the token doesn't list "survey"). This ensures security boundaries.
26. **Documentation and Onboarding:** Write documentation for the new service so others know how to use its API and UI. Because we have a modular structure, adding new services follows a predictable pattern, which lowers the learning curve for new team members or contributors.

By following these steps, the architecture remains **extensible**. New services can be added without touching the internals of existing services (except maybe the core for auth config). This plug-and-play ability is a hallmark of good microservice design and will help the system evolve into a suite of tools (form builder, short links, surveys, etc.) under one integrated platform.

Conclusion

In summary, this blueprint outlines a **scalable microservice architecture** using Node.js, Express, Angular, PostgreSQL, Kafka, and Redis, with JWT-based SSO authentication and multi-tenancy support. The design emphasizes separation of concerns: each microservice handles a specific business capability and has its

own data, while a central core service manages authentication and high-level coordination. By using JWT tokens for cross-service auth, we achieve a seamless SSO experience for users across the different services ⁴. The inclusion of Kafka messaging enables an event-driven approach that improves system resilience and scalability by decoupling services ⁵. Redis provides caching and fast data access where needed, boosting performance for frequently accessed resources ⁶.

We've also considered the operational aspects by planning for Docker-based deployment and CI/CD automation from the get-go, making sure the system can be easily deployed in containers and updated continuously. The architecture supports both single-tenant and multi-tenant modes through configurable tenancy isolation, following best practices to prevent data leakage between tenants ³.

By adhering to this roadmap, we can develop the project in stages, validate at each step, and remain flexible to add new features as separate services down the line. This approach ensures that our platform can grow into a robust suite of microservices, all integrated under a unified security and user experience layer, ready for production deployment and scaling.

Lastly, throughout the development, we will keep in mind best practices and modern patterns (like API Gateway usage, BFF, etc.) ⁸, even if not all are fully implemented initially, so that the architecture can evolve (for example, introducing an OAuth2/OIDC server in the future or splitting services further if needed). This blueprint serves as a **canvas** for building the system, providing a clear structure and path forward for the engineering team. With this foundation, we can confidently proceed to implement the project, knowing that the architecture will support our needs for modularity, scalability, and maintainability.

Sources:

- Microservice architecture components and best practices ¹ ²⁶
- JWT-based auth for microservices and SSO design ⁴ ⁵
- Multi-tenant SaaS patterns and tech stack (Node.js, PostgreSQL, JWT, Redis) ¹³ ⁶
- Multi-tenancy data isolation approaches (shared DB vs schema vs separate DB) ³ ¹⁷
- Kafka in microservices for decoupling and scalability ⁵
- Typical Node.js microservice project structure and considerations ²⁷ ²
- JWT usage in stateless auth (tokens as self-contained credentials) ¹² and avoiding server-side token store

¹ ² ²² ²³ ²⁶ ²⁷ Node.js microservice architecture - DEV Community

<https://dev.to/krishna7852/nodejs-microservice-architecture-58lp>

³ ⁶ ¹³ Build Multi-Tenant SaaS Boilerplate in Node.js [With GitHub Code]

<https://www.sevensquaretech.com/multi-tenant-saas-boilerplate-nodejs-with-github-code/>

⁴ ⁸ ¹⁰ ¹¹ ¹⁴ ¹⁵ Authentication and authorization in a microservice architecture: Part 2 - Authentication

<https://microservices.io/post/architecture/2025/05/28/microservices-authn-authz-part-2-authentication.html>

⁵ ⁹ ²⁰ ²¹ ²⁵ Building a Scalable Microservices Architecture with Kafka and Express.js | by Mahmudur Rahman | Medium

<https://medium.com/@hridoymahmud/building-a-scalable-microservices-architecture-with-kafka-and-express-js-4f5c800dfbe6>

7 24 Building a Scalable Multi-Tenant Architecture in Express.js with Mongoose | by MUHAMMED SHAFEEQUE P | Jul, 2025 | Medium

<https://medium.com/@shafeequekv95/building-a-scalable-multi-tenant-architecture-in-express-js-with-mongoose-b383be13e6fe>

12 php - JWT Authentication within a Micro Service architecture - Stack Overflow

<https://stackoverflow.com/questions/35639882/jwt-authentication-within-a-micro-service-architecture>

16 17 18 19 Complete Guide to Multi-Tenant Architecture | by Seetharamugn | Medium

<https://medium.com/@seetharamugn/complete-guide-to-multi-tenant-architecture-d69b24b518d6>